ΕΙΔΙΚΑ ΘΕΜΑΤΑ ΤΕΧΝΟΛΟΓΙΑΣ ΛΟΓΙΣΜΙΚΟΥ

ΣΥΣΤΗΜΑ ΔΙΑΧΕΙΡΙΣΗΣ ΡΑΝΤΕΒΟΥ

ΠΑΝΤΕΛΕΗΜΩΝ ΚΑΡΑΜΠΕΤΣΟΣ

Α.Μ: 20390089 | 13/12/2025

1. Εισαγωγή

Στην παρούσα εργασία υλοποιήσαμε μια web εφαρμογή διαχείρισης ραντεβού ιατρείου με τίτλο **Clinic Appointment System**.
Η εφαρμογή απευθύνεται στη γραμματεία/διαχειριστή του ιατρείου και στόχος της είναι να δίνει ένα απλό περιβάλλον για:

- καταχώρηση και διαχείριση ασθενών,
- προγραμματισμό και παρακολούθηση ραντεβού,
- βασική εικόνα της λειτουργίας μέσα από ένα dashboard.

Η εφαρμογή ακολουθεί αρχιτεκτονική 3 επιπέδων (front-end / business logic / database). Το UI υλοποιήθηκε ως SPA με Next.js & React, το backend (REST API) μέσω Next.js (Node.js), ενώ τα δεδομένα αποθηκεύονται σε PostgreSQL. Η επικοινωνία με τη βάση γίνεται μέσω ORM (Sequelize).

Επίσης οργανώσαμε την ανάπτυξη με agile λογική, χρησιμοποιώντας User Stories, Product Backlog, sprints και Kanban board στο Trello, ώστε να υπάρχει καθαρή εικόνα της προόδου.

2. Μεθοδολογία Ανάπτυξης

Για την ανάπτυξη ακολουθήσαμε μια απλή agile προσέγγιση, βασισμένη στη λογική του Scrum.

- Οι απαιτήσεις γράφτηκαν ως User Stories (π.χ. “Ως διαχειριστής θέλω…”).
- Οι ιστορίες ομαδοποιήθηκαν σε Epics.
- Όλα συγκεντρώθηκαν σε ένα Product Backlog.
- Η υλοποίηση έγινε σε sprints διάρκειας ~15 ημερών.

2.1. Λόγοι επιλογής agile προσέγγισης

Η agile προσέγγιση μας βοήθησε:

- να υλοποιούμε λειτουργίες βήμα βήμα,
- να βάζουμε προτεραιότητες (π.χ. πρώτα login, μετά ασθενείς, μετά ραντεβού),
- να έχουμε πιο εύκολα έλεγχο προόδου και βελτίωση των λειτουργιών.

Κάθε User Story συνοδεύεται από Acceptance Criteria για να είναι ξεκάθαρο πότε θεωρείται ολοκληρωμένη.

2.2. Product Backlog

Το backlog οργανώθηκε στα παρακάτω Epics:

- **EPIC 1:** Authentication & περιορισμένη πρόσβαση (admin area)

- **EPIC 2:** Διαχείριση ασθενών
- **EPIC 3:** Διαχείριση ραντεβού
- **EPIC 4:** Dashboard (πίνακας διαχείρισης)
- **EPIC 5:** Διαχείριση χρηστών συστήματος (βασικά)

  Τα User Stories είναι κατανεμημένα από US-01 έως US-12.

2.3. Sprints

Η ανάπτυξη οργανώθηκε ως εξής:

Sprint 1

- Υλοποίηση Next.js project και βασικής δομής.
- Υλοποίηση σύνδεσης Login (username/password) και δημιουργία JWT.
- Βασική δομή μοντέλων στη βάση (User, Patient).

Sprint 2

- Ολοκληρωμένη λειτουργία διαχείρισης ασθενών (CRUD, λίστα, αναζήτηση, επεξεργασία).
- Βελτιώσεις στην σελίδα ασθενών και στο UI.

Sprint 3

- Δημιουργία σελίδας διαχείρισης ραντεβού (δημιουργία, λίστα, φιλτράρισμα).
- Συσχέτιση των ραντεβού με ασθενείς (Appointment → Patient).

Sprint 4

- Ανάπτυξη **Dashboard** (πίνακα διαχείρισης).
- Τελικές βελτιώσεις στο UI.
- Testing με Postman και εντοπισμός σφαλμάτων.

2.4. Board στο Trello

Για την οργάνωση των tasks χρησιμοποιήσαμε ένα board στο Trello, με τις στήλες:

- Backlog
- To Do
- In Progress

- Testing
- Done

Κάθε εργασία προστέθηκε ως κάρτα, με μια σύντομη περιγραφή και σημειώσεις.

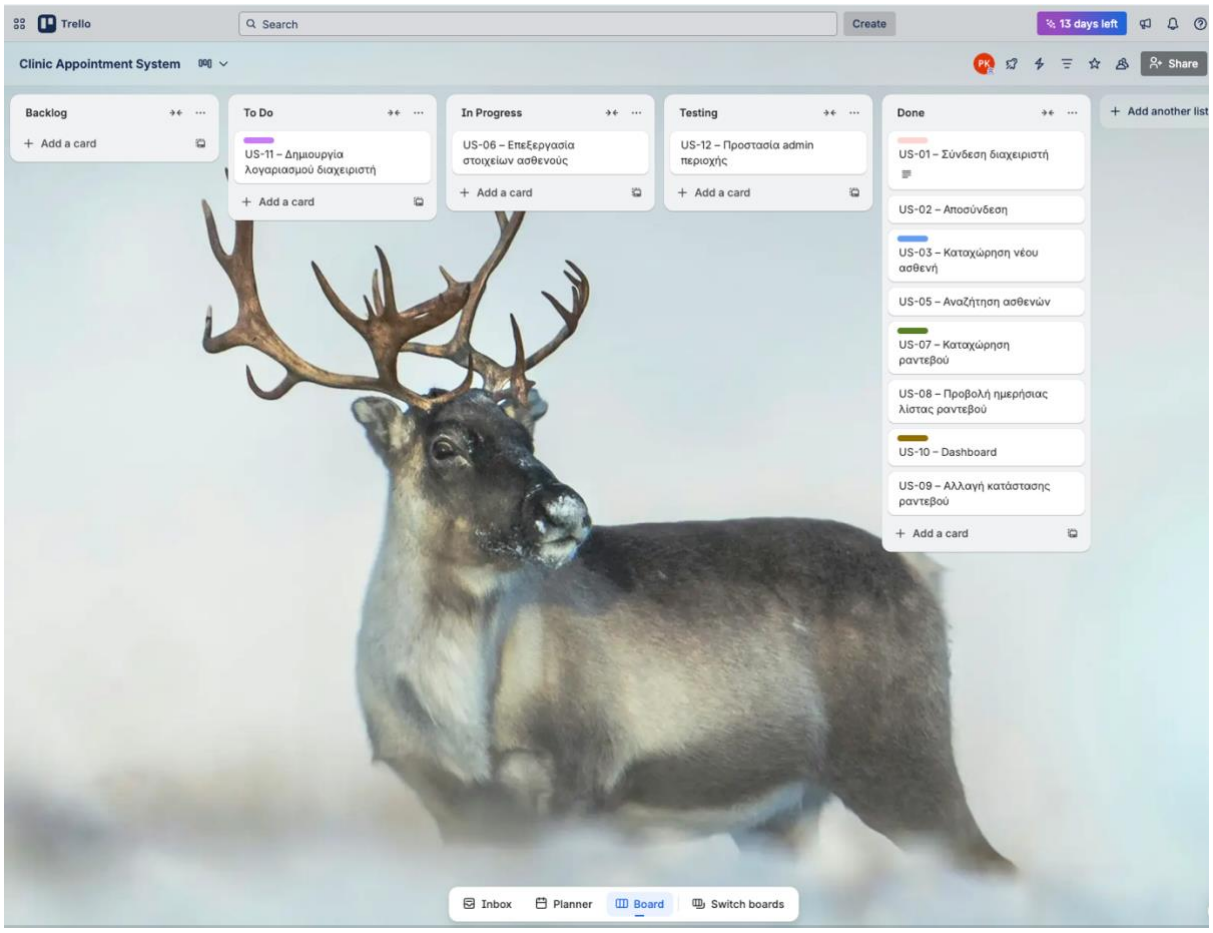

Figure 1: board στο Trello (οργάνωση εργασιών ανα status)

3. Ρόλοι Χρηστών

Η εφαρμογή έχει έναν βασικό χρήστη:

3.1.Διαχειριστής

Ο ρόλος αυτός αφορά το προσωπικό του ιατρείου (π.χ γραμματεία ή ιατρός). Μπορεί να:

- συνδέεται στο σύστημα με username και password,
- καταχωρεί, επεξεργάζεται και να βλέπει στοιχεία ασθενών,
- προγραμματίζει, τροποποιεί και να παρακολουθεί τα ραντεβού,
- έχει πρόσβαση στο dashboard.

Σημείωση: Η εφαρμογή προστατεύει τις σελίδες περιορισμένης πρόσβασης (admin). Δεν χρησιμοποιήθηκαν πολλαπλοί ρόλοι (π.χ. διαφορετικά permissions ανά ρόλο), αλλά υπάρχει έλεγχος πρόσβασης μέσω του login page.

4. Τεχνολογικό Stack & Αρχιτεκτονική Συστήματος

Η εφαρμογή Clinic Appointment System ακολουθεί 3-tier αρχιτεκτονική:

1. Front end (UI)
2. Backend (API, Bussiness logic)
3. Database (PostgreSQL με ORM)

4.1. Επιλογή τεχνολογικού stack

Το συνολικό stack που χρησιμοποιήθηκε είναι:

- *Front-end*: Next.js, React

  Σελίδες: /login, /dashboard, /patients, /appointments
- *Backend*: Node.js

  Endpoints: /api/auth/login, /api/patients, /api/appointments, /api/users
- *Database*: PostgreSQL

  ORM: Sequelize
- *Auth*: JWT (token αποθηκεύεται στο browser)

4.2. Front-end

σελίδες:

- *Login (/login):* φόρμα username/password, κλήση /api/auth/login, αποθήκευση token, redirect σε dashboard.
- *Dashboard (/dashboard):* Πίνακας διαχείρισης και κάποια μικρά στατιστικά (π.χ σύνολο ασθενών, συνολικά ραντεβού).
- *Patients (/patients):* σελίδα για καταχώρηση ασθενούς, λίστα με όλους τους ασθενείς, αναζήτηση, επεξεργασία, διαγραφή ασθενή.
- *Appointments (/appointments):* σελίδα για δημιουργία ραντεβού, λίστα ραντεβού με αναζήτηση και φίλτρα.

4.3. Backend (API, Bussiness logic)

Η επιχειρησιακή λογική υλοποιείται με Node.js routes:

- **/api/auth/login**
  - Παραλαμβάνει username, password.
  - Ελέγχει τα credentials μέσω του μοντέλου User.
  - Με την επιτυχής σύνδεση δημιουργεί JWT token και το επιστρέφει πίσω.
- **/api/users**
  - Δημιουργεί νέους χρήστες στο σύστημα.
  - Ο κωδικός αποθηκεύεται με ασφάλεια σε hash.
- **/api/patients**
  - GET: επιστρέφει λίστα ασθενών.
  - POST: δημιουργεί νέο ασθενή.
  - /api/patients/[id] (PUT): επεξεργασία στοιχείων ασθενή.
  - /api/patients/[id] (DELETE): διαγραφή ασθενή.
- **/api/appointments**
  - GET: επιστρέφει όλα τα ραντεβού ταξινομημένα κατά dateTime, μαζί με τα στοιχεία ασθενούς (μέσω include στο Sequelize).
  - POST: δημιουργεί νέο ραντεβού.
- **/api/appointments/[id]**
  - PUT: ενημέρωση στοιχείων ραντεβού (ημερομηνία, ώρα, διάρκεια, status).
  - DELETE: διαγραφή ραντεβού.

Κάθε route ακολουθεί:

1. Κλήση initDb() για την σύνδεση με τη βάση δεδομένων και συγχρονισμό μοντέλων.
2. Έλεγχος στοιχείων εισόδου.
3. Χρήση των Sequelize models (User, Patient, Appointment) για ενέργειες.
4. Επιστροφή API response (JSON) με HTTP status codes.

4.4. Βάση Δεδομένων

Χρησιμοποιήθηκε PostgreSQL με τους πίνακες:

- Users
- Patients
- Appointments (συσχετισμένα με ασθενείς)

Με την Sequelize ορίστηκαν τα μοντέλα και οι σχέσεις (π.χ. Appointment → Patient). Η σύνδεση γίνεται με environment variables.

4.5. Authentication & Προστασία Admin περιοχής

Η αυθεντικοποίηση γίνεται με JWT:

- Στο login, αν τα στοιχεία είναι σωστά, το backend επιστρέφει το token.
- Το frontend αποθηκεύει το token και το στέλνει σε μελλοντικά requests.
- Αν δεν υπάρχει token, το API επιστρέφει **401** και το frontend κάνει redirect στο /login.

4.6.Dependency Injection & οργάνωση κώδικα

Στο project οργανώσαμε τον κώδικα ώστε οι βασικές συσχετίσεις να δημιουργούνται κεντρικά και να επαναχρησιμοποιούνται στα endpoints, αντί να δημιουργούνται κάθε φορά από την αρχή.

- Η ρύθμιση του Sequelize και των models γίνεται σε κοινά modules.
- Τα API routes χρησιμοποιούν αυτά τα modules για να εκτελούν λειτουργίες.

5. Testing (Postman)

Στόχος του testing είναι να εξασφαλίσει ότι τα βασικά σενάρια χρήσης (login, διαχείριση ασθενών, διαχείριση ραντεβού) λειτουργούν σωστά.

Επειδή η εφαρμογή είναι κυρίως CRUD, δόθηκε έμφαση σε API tests.

5.1. Εργαλείο Testing

Για το testing χρησιμοποίησα το εργαλείο Postman όπου δημιούργησα collection που περιλαμβάνει πολλαπλά requests για να είναι ρεαλιστικό.

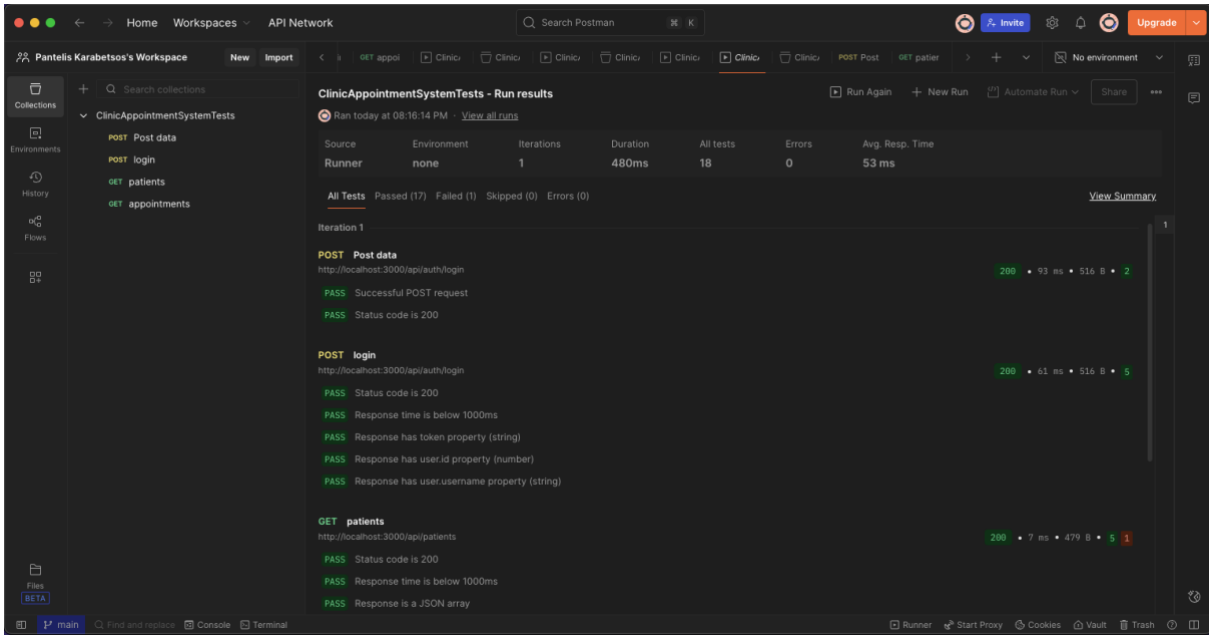

Figure 2: Postman Collection (API & integration testing) για τα endpoints της εφαρμογής.

5.2. Σενάρια ελέγχου

Μέσα από τα requests του Postman επιβεβαιώσαμε:

- Επιστρέφονται τα αναμενόμενα status codes,
- Επιστρέφεται επιτυχώς το token μετά το login,
- Όλες οι λειτουργίες για τα δεδομένα μας λειτουργούν σωστά(δημιουργία, fetch, διαγραφή, επεξεργασία)
- Ανάλογη προστασία των endpoints όταν δεν υπάρχει token.

Τα σενάρια ελέγχου ήταν τα παρακάτω:

- **TS-01:** Επιτυχές login → επιστρέφεται JWT token.
- **TS-02:** Login με λάθος στοιχεία → επιστρέφεται κωδικός σφάλματος (401/400).
- **TS-03:** Δημιουργία νέου ασθενούς → επιστρέφεται κωδικός επιτυχίας με δεδομένα (201).
- **TS-04:** Δημιουργία ασθενούς με ελλιπή στοιχεία → κωδικός σφάλματος (400).
- **TS-05:** Δημιουργία ραντεβού για υπάρχον ασθενή → κωδικός επιτυχίας (201).
- **TS-06:** Ενημέρωση κατάστασης ραντεβού → κωδικός επιτυχίας (200).
- **TS-07:** Προσπάθεια πρόσβασης σε προστατευμένο endpoint χωρίς token → κωδικός σφάλματος (401).

5.3. Συμπέρασμα

Με τα παραπάνω tests επιβεβαιώθηκε ότι:

- η αυθεντικοποίηση λειτουργεί σωστά,
- τα βασικά CRUD για ασθενείς και ραντεβού είναι σταθερά,
- τα προστατευμένα endpoints δεν είναι διαθέσιμα χωρίς σύνδεση διαχειριστή,
- τα δεδομένα αποθηκεύονται και ανακτώνται σωστά από τη βάση

6. Epics (Ομαδοποίηση User Stories)

- EPIC 1: Authentication & προστασία πρόσβασης
- EPIC 2: Διαχείριση ασθενών
- EPIC 3: Διαχείριση ραντεβού
- EPIC 4: Dashboard
- EPIC 5: Διαχείριση χρηστών συστήματος

7. User Stories

Στην ενότητα αυτή παρουσιάζονται συνοπτικά τα User Stories που υλοποιήθηκαν. Κάθε ιστορία περιγράφει τι χρειάζεται ο χρήστης και πότε θεωρείται ολοκληρωμένη (κριτήρια αποδοχής).

EPIC 1: Authentication & Πρόσβαση (βασικό RBAC)

US-01: Σύνδεση διαχειριστή

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να μπορώ να συνδέομαι με username και password, ώστε να έχω ασφαλή πρόσβαση στις λειτουργίες του ιατρείου.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

⇒ Υπάρχει σελίδα login με πεδία username και password.
⇒ Με σωστά στοιχεία γίνεται login, επιστρέφεται JWT token και ο χρήστης μεταφέρεται στο dashboard.
⇒ Με λάθος στοιχεία εμφανίζεται μήνυμα σφάλματος.
⇒ Οι προστατευμένες σελίδες δεν είναι προσβάσιμες χωρίς έγκυρο token.

US-02: Αποσύνδεση (Logout)

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να μπορώ να κάνω logout, ώστε να μη μείνει ενεργή η πρόσβαση στον ίδιο υπολογιστή.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

- ⇒ Υπάρχει κουμπί «Αποσύνδεση» στο header.
- ⇒ Με το logout διαγράφεται το JWT token από το localStorage.
- ⇒ Μετά την αποσύνδεση ο χρήστης επιστρέφει στη σελίδα login.
- ⇒ Απόπειρα πρόσβασης σε προστατευμένη σελίδα μετά το logout οδηγεί σε redirect στο login.

EPIC 2: Διαχείριση Ασθενών

US-03: Καταχώρηση νέου ασθενούς

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να καταχωρώ έναν νέο ασθενή, ώστε να δημιουργείται φάκελος ασθενούς στο σύστημα.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

- ⇒ Υπάρχει φόρμα «Νέος ασθενής» με τουλάχιστον: Όνομα, Επώνυμο, Τηλέφωνο, Email, Ημ/νία γέννησης, Σημειώσεις.
- ⇒ Τα υποχρεωτικά πεδία ελέγχονται (validation).
- ⇒ Σε επιτυχή καταχώρηση εμφανίζεται μήνυμα επιτυχίας.
- ⇒ Ο νέος ασθενής εμφανίζεται άμεσα στη λίστα.

US-04: Προβολή λίστας ασθενών

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να βλέπω τη λίστα όλων των ασθενών, ώστε να εντοπίζω γρήγορα τα στοιχεία τους.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

Η λίστα περιλαμβάνει: Ονοματεπώνυμο, Τηλέφωνο, Email, Ημ/νία δημιουργίας.

Οι ασθενείς εμφανίζονται ταξινομημένοι (π.χ. οι πιο πρόσφατοι πρώτοι).

Η λίστα ενημερώνεται σωστά μετά από νέα καταχώρηση.

US-05: Αναζήτηση ασθενών

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να μπορώ να αναζητώ ασθενείς με βάση όνομα, επώνυμο ή τηλέφωνο, ώστε να βρίσκω γρήγορα αυτόν που χρειάζομαι.

Προτεραιότητα: Μεσαία

Κριτήρια αποδοχής:

Υπάρχει πεδίο αναζήτησης πάνω από τη λίστα.

Καθώς ο χρήστης πληκτρολογεί, η λίστα φιλτράρεται.

Η αναζήτηση εφαρμόζεται τουλάχιστον σε ονοματεπώνυμο και τηλέφωνο.

US-06: Επεξεργασία στοιχείων ασθενούς

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να μπορώ να ενημερώνω τα στοιχεία ενός ασθενούς (π.χ. τηλέφωνο, email), ώστε τα δεδομένα να παραμένουν σωστά και ενημερωμένα.

Προτεραιότητα: Μεσαία

Κριτήρια αποδοχής:

- ⇒ Από τη λίστα ασθενών ανοίγει φόρμα επεξεργασίας.
- ⇒ Τα υπάρχοντα στοιχεία φορτώνονται στη φόρμα.
- ⇒ Μετά την αποθήκευση, η λίστα ενημερώνεται.
- ⇒ Γίνεται validation όπως και στη δημιουργία ασθενούς.

EPIC 3: Διαχείριση Ραντεβού

US-07: Καταχώρηση ραντεβού για υπάρχοντα ασθενή

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να προγραμματίζω ραντεβού για έναν ήδη καταχωρημένο ασθενή, ώστε να οργανώνεται σωστά το πρόγραμμα του ιατρείου.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

⇒ Υπάρχει φόρμα «Νέο ραντεβού» με επιλογή ασθενούς.
⇒ Καταχωρούνται: ημερομηνία, ώρα, λόγος επίσκεψης και προαιρετικές σημειώσεις.
⇒ Το ραντεβού αποθηκεύεται στη βάση και συνδέεται με τον ασθενή.
⇒ Εμφανίζεται μήνυμα επιτυχίας μετά την καταχώρηση.

US-08: Προβολή ημερήσιας λίστας ραντεβού

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να βλέπω τα ραντεβού μιας συγκεκριμένης ημερομηνίας, ώστε να γνωρίζω το πρόγραμμα της ημέρας.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

⇒ Υπάρχει επιλογή ημερομηνίας (date picker).
⇒ Για την επιλεγμένη ημερομηνία εμφανίζονται: ώρα, ασθενής, τηλέφωνο, λόγος, κατάσταση.
⇒ Τα ραντεβού είναι ταξινομημένα ανά ώρα.

US-09: Αλλαγή κατάστασης ραντεβού

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω να μπορώ να αλλάζω την κατάσταση ενός ραντεβού, ώστε να παρακολουθώ την εξέλιξή του (π.χ. ολοκληρώθηκε/ακυρώθηκε).

Προτεραιότητα: Μεσαία

Κριτήρια αποδοχής:

⇒ Στη λίστα υπάρχει επιλογή αλλαγής κατάστασης.
⇒ Οι επιτρεπόμενες τιμές είναι: scheduled, completed, cancelled.
⇒ Η αλλαγή αποθηκεύεται στη βάση και εμφανίζεται άμεσα στο UI.

EPIC 4: Dashboard & Επισκόπηση

US-10: Βασικό dashboard επισκόπησης

Ρόλος: Διαχειριστής & Γραμματεία

Περιγραφή: Ως διαχειριστής, θέλω στο dashboard να βλέπω βασικά στατιστικά, ώστε να έχω γρήγορη εικόνα της κατάστασης.

Προτεραιότητα: Μεσαία

Κριτήρια αποδοχής:

- ⇒ Εμφανίζεται κάρτα με το σύνολο των ασθενών.
- ⇒ Εμφανίζεται κάρτα με τον αριθμό ραντεβού (π.χ. συνολικά ή για σήμερα).
- ⇒ Εμφανίζεται ο τελευταίος καταχωρημένος ασθενής με ημερομηνία δημιουργίας.
- ⇒ Σε σφάλμα φόρτωσης εμφανίζεται κατάλληλο μήνυμα.

EPIC 5: Διαχείριση Χρηστών & Ασφάλεια

US-11: Δημιουργία λογαριασμού διαχειριστή

Ρόλος: Διαχειριστής συστήματος & Ανάπτυξη

Περιγραφή: Ως υπεύθυνος του συστήματος, θέλω να μπορώ να δημιουργώ λογαριασμό διαχειριστή με username και password, ώστε να δίνεται πρόσβαση στη γραμματεία/ιατρό.

Προτεραιότητα: Μεσαία

Κριτήρια αποδοχής:

- ⇒ Υπάρχει endpoint για δημιουργία χρήστη.
- ⇒ Ο κωδικός αποθηκεύεται με ασφάλεια με μορφή hash.
- ⇒ Κάθε username είναι μοναδικό, δεν επιτρέπεται διπλότυπο.
- ⇒ Μετά τη δημιουργία, ο χρήστης μπορεί να συνδεθεί από τη σελίδα login.

US-12: Προστασία admin περιοχής

Ρόλος: Σύστημα

Περιγραφή: Ως σύστημα, πρέπει να επιτρέπω πρόσβαση στις σελίδες διαχείρισης μόνο σε αυθεντικοποιημένους χρήστες, ώστε να προστατεύονται τα δεδομένα του ιατρείου.

Προτεραιότητα: Υψηλή

Κριτήρια αποδοχής:

- ⇒ Οι σελίδες dashboard, patients και appointments απαιτούν έγκυρο JWT token.
- ⇒ Αν το token λείπει ή είναι άκυρο, τα API endpoints επιστρέφουν 401 και το UI κάνει redirect στο login.
- ⇒ Τα σφάλματα αυθεντικοποίησης χειρίζονται σωστά στο front-end.

8. Αποθετήριο κώδικα & Διαχείριση Εκδόσεων

Για version control χρησιμοποιήθηκε Git και ο κώδικας ανέβηκε σε ιδιωτικό GitHub repository.
Κατά την ανάπτυξη έγιναν commits με περιγραφή, ώστε να υπάρχει ιστορικό επεξεργασίας.

9. Συμπεράσματα

Για την εργασία αυτή υλοποιήσαμε μια CRUD web εφαρμογή για διαχείριση ραντεβού και ασθενών ενός μικρού ιατρείου :

- 3-tier αρχιτεκτονική (UI / API / DB)
- SPA frontend με Next.js & React
- REST API με Node.js
- PostgreSQL και πρόσβαση μέσω ORM (Sequelize)
- Authentication με JWT και προστασία περιορισμένων περιοχών (admin)
- agile οργάνωση με User Stories, backlog, sprints και Trello board
- integration testing σε API με Postman.